main() Method in Java.

✓ What is the main() Method in Java?

The **main() method** is the **entry point** of any standalone Java application.
When you run a Java program, the JVM looks for the main() method and starts executing your code from there.

🔥 Official Method Signature

```
public static void main(String[] args)
```

Let's break it down **word by word**:

---

1 public

- It is an **access modifier**.
- JVM must access this method **from outside the class**, so it must be **public**.
- If you make it private → program will compile but **not run**.

---

2 static

- main() belongs to the **class**, not the object.
- JVM should run the program **without creating an object**, so it must be static.
- Static ensures:
  - Fast loading
  - No object needed
  - Executes at class loading time

---

3 void

- main() does **not return any value** to the JVM.
- Its job is only to **start the program**, not return a value.

---

4 main

- This is the **predefined name** JVM looks for.
- If you change the method name → JVM will not recognize it → program will not run.

---

5 String[] args

- It is the **command-line argument array**.
- Anything you pass while running the program goes into this array.

Example:

```
java MyClass hello 123

args[0] = "hello"
args[1] = "123"
```

You can also write:

- String args[]
- String... args (var-args)

All are valid.

🧠 Why is main() important?

Because JVM uses it as:

✓ The **starting point** of your program
✓ To **load the class**
✓ To bring your application's logic to life
✓ To execute your program **step by step** from here

If there is **no main()**, JVM throws:

```
Error: Main method not found in class...
```

🧪 Working Example

```
public class Demo {

    public static void main(String[] args) {
        System.out.println("Program started!");
    }
}
```

🔄 Can we overload main()?

Yes, you can overload:

```
public static void main(int a) { }
```

But JVM will call **only**:

```
public static void main(String[] args)
```

Other overloaded methods **won't be executed automatically**.

📌 Can we change order?

Yes, these are valid:

- static public void main(String[] args)
- public static void main(String[] args)
- public static void main(String args[])

But this is preferred:

```
public static void main(String[] args)
```

🎤 Interview-Ready Final Answer

**The main() method is the entry point of a Java application.
It is defined as `public static void main(String[] args)` because JVM needs to access it globally (public), execute it without an object (static), and it doesn't return anything (void). The method receives command-line arguments through the String array. JVM begins program execution from this method. Without main(), no standalone Java program can run.**

✅ How many ways can we write the main() method in Java?

There are **6 valid ways** to write the main method (all accepted by JVM).
The core rule: **signature must stay the same**, but order and array syntax can change.

✅ 6 Valid Ways to Write main()

1. Standard way (preferred)
```
public static void main(String[] args)
```

2. Changing order of modifiers
```
static public void main(String[] args)
```

3. Changing array style
```
public static void main(String args[])
```

4. Another array syntax variation
```
public static void main(String []args)
```

5. Using var-args (valid for JVM)
```
public static void main(String... args)
```

6. Static + public order change with var-args
```
static public void main(String... args)
```

❌ Invalid Ways (JVM will NOT run)

❌ return type change
```
public static int main(String[] args)   // invalid
```

❌ missing static
```
public void main(String[] args)   // invalid
```

❌ wrong parameter type
```
public static void main(int[] args)   // invalid
```

✅ Compile-Time Errors for main() Method

These errors happen **before running** the program — during compilation.

---

1 Wrong method signature

public static int main(String[] args)

❌ Compile-time error:

Error: invalid method declaration; return type required

---

2 Missing static keyword

public void main(String[] args)

✓ Compiles? ❌ No, it compiles but JVM won't run it.
However, technically the *signature is wrong*, so compiler will not complain, but JVM gives error at runtime (explained later).

---

3 Wrong parameter type

public static void main(int[] args)

❌ Compiles? Yes.
BUT **JVM won't treat this as main** → runtime error (explained below).

---

4 Wrong number of arguments

public static void main(String args, int a)

❌ Compile-time error.

---

5 Wrong access modifier

private static void main(String[] args)

✓ Compiles fine.
But at runtime: JVM error.
(Not compile-time, but important for interview.)

---

6 Syntax errors around main

public static void main(String[] args {

✗ Compile-time error due to missing bracket.

🚨 Summary (Compile-Time Errors Only)

Compile-time errors occur when:

✓ Wrong syntax
✓ Invalid modifiers
✓ Wrong parameters
✓ Wrong brackets / missing return type / type mismatch

🔥 Runtime Errors for main() Method

These errors happen **after successful compilation**, when JVM tries to run the program.

---

1 Main method not found

public void main(String[] args)

or

public static void main(int[] args)

✗ Runtime error:

Error: Main method not found in class Demo

---

2 Main method is not public

static void main(String[] args)

or

private static void main(String[] args)

❌ Runtime error:

Error: Main method not found in class Demo, please define the main method as:
public static void main(String[] args)

---

3 Main method is not static

public void main(String[] args)

❌ Runtime error:

Error: Main method must be static

---

4 Wrong signature even if overloaded

public static void main(String args) // wrong

❌ Runtime error:

Error: Main method not found...

✅ All Possible main() Method Syntax Cases — Corrected & Explained

---

1 public static void main(String[] args) – Correct

This is the **standard** and **recommended** main method.
JVM looks for **exactly this signature** (order of modifiers can change).

---

2 public static void main(Integer[] args) – Method correct, main() incorrect

- This is **valid Java method syntax**, but
- JVM does **NOT** accept `Integer[]` as the parameter type.
  JVM only recognizes:

✓ `String[] args`
✓ `String args[]`
✓ `String... args`

So this version **compiles** but **JVM won't treat it as main()**.

---

3 static public void main(String[] args) – Correct

Order of modifiers **does not matter**.

Java allows:

- public static
- static public

Both are valid.

---

4 public void static main(String[] args) – Incorrect (Compile-time error)

This is invalid **method syntax**.

Why?

Java method syntax rule:

<modifiers> <return-type> <method-name>(arguments)

Here:

- void must come **before method name**
- But static is placed **after** void, which is not allowed.

Hence → **compile-time error**.

---

5 public static void args(String[] args) or public static void xyz(String[] args) – Method correct, main() incorrect

These are **perfectly valid methods**, but not acceptable as JVM entry points.

JVM requires the name **main** only.

---

6 public static void main(String[ ] args) – Correct

Extra space before brackets → **valid**.

---

7 public static void main(String [] args) – Correct

Space between type and bracket → allowed.

---

8 public static void main(String []args) – Correct

Bracket attached to variable → also valid.

---

9 public static void main(String args[]) – Correct

Alternative array syntax → JVM accepts it.

---

10 public static void main(String args() {}) – Incorrect

Because:

- `args()` is function syntax
- `{}` is initializer block
- Not allowed as a variable declaration

Invalid → **compile-time error**.

---

11 public static void main(String args) – Method correct, main() incorrect

This is valid Java method, but:

❌ JVM does NOT accept a **single String** argument.
✓ Must be a **String array**.

---

12 public static void main(String[] xyz) – Correct

Parameter name doesn't matter.

✓ args
✓ xyz
✓ anything is allowed.

---

13 public static void main(String... xyz) – Correct

Var-args version of main.

✓ JVM accepts String... args
⚠ Must have **exactly 3 dots**: ....

---

14 protected/private static void main(String[] args) – Method correct, main() incorrect

Technically, the method **compiles**, BUT:

✗ JVM needs **public** access
Otherwise, runtime error:

Error: Main method not found...

---

15 final public static void main(String[] args) – Correct

You can add final.
JVM accepts this.

---

16 synchronized public static void main(String[] args) – Correct

synchronized is allowed as an additional modifier.

✓ Valid
✓ JVM accepts it
Though it's not recommended in practice.

🔘 Final Summary Table (Easy to Learn)

Syntax	Valid Method?	Valid main() for JVM?	Notes
public static void main(String[] args)	✓	✓	Standard
String args[] / String[] args / String []args	✓	✓	All accepted
String... args	✓	✓	Var-args accepted
Modifier order changed	✓	✓	e.g., static public
Extra modifiers (final/synchronized)	✓	✓	JVM accepts
Integer[] args	✓	✗	JVM rejects
Wrong method name	✓	✗	JVM rejects
Wrong access modifier (private/protected)	✓	✗	Runtime error
Wrong return type	✗	✗	Compile-time error
Wrong syntax (args(), {}, missing brackets)	✗	✗	Compile-time error

---

✅ When main() Method Is NOT Required in Java

The main() method is **not always required**. It is needed **only for standalone execution.**

🔻 main() is NOT required in these cases:

1 When running code inside another application

- Spring Boot
- Web applications
- Application servers (Tomcat, JBoss)

➡ Framework provides its own entry point.

---

2 For classes used as helpers / utilities

```
class MathUtil {
    int add(int a, int b) {
        return a + b;
    }
}
```

➞ Used by other classes, not run directly.

---

3 When using JUnit / Test classes

```
@Test
void testAdd() {
    assertEquals(5, add(2,3));
}
```

➞ Test runner executes tests, not main().

---

4 Abstract classes & Interfaces

- Cannot be instantiated directly
- So main() is unnecessary

---

5 Applets (legacy – deprecated)

- Used lifecycle methods like init() and start()
- No main() needed

❌ When main() IS required

✓ Standalone Java application
✓ Program executed directly using JVM
✓ Basic Java programs

💡 Interview One-Liner

main() method is required **only when a class needs to be executed directly by JVM**, otherwise not.